

Capítulo 1

Estructura de la aplicación web propuesta

1.1. Introducción

La aplicación desarrollada en el presente trabajo corresponde a una plataforma web geoespacial orientada a la visualización, carga, edición, consulta, descarga y administración de información vinculada al análisis de la calidad del agua subterránea en el acuífero Patiño. Desde una perspectiva funcional, la plataforma combina elementos de un visor cartográfico web, un sistema de administración de datos geoespaciales, un mecanismo de colaboración multiusuario y un conjunto de procesos de transformación de datos para interoperar con distintos formatos de entrada y salida.

La estructura de la aplicación no responde a un diseño monolítico puramente estático, sino a una arquitectura de varias capas que integra una interfaz web interactiva, un backend transaccional implementado sobre el marco Django, una base de datos espacial PostGIS y un conjunto de herramientas SIG externas para el procesamiento de archivos vectoriales y raster. Este enfoque permite que la solución no se limite a la simple publicación de mapas, sino que incorpore capacidades de gestión y validación de datos, control de permisos, trazabilidad de cambios, importación de datos de campo y publicación diferenciada de contenidos.

En términos conceptuales, la plataforma fue diseñada para soportar tres necesidades principales: en primer lugar, la representación visual del territorio y de los puntos de muestreo; en segundo lugar, la administración estructurada de capas y grupos de datos por parte de distintos tipos de usuarios; y, en tercer lugar, la normalización de datos heterogéneos provenientes de campañas de muestreo, archivos CSV, GeoJSON, Shapefile y GeoTIFF. La integración

de estas necesidades da lugar a una estructura de aplicación que debe ser comprendida tanto desde el punto de vista tecnológico como desde el punto de vista funcional.

1.2. Propósito general de la arquitectura

La arquitectura de la aplicación fue concebida con el objetivo de mantener un equilibrio entre flexibilidad funcional, reutilización de componentes, interoperabilidad geoespacial y sostenibilidad técnica. En lugar de depender de una plataforma SIG cerrada, la solución se construyó sobre herramientas de código abierto ampliamente difundidas, lo que facilita su mantenimiento, extensión y posible adaptación a otros escenarios académicos o institucionales.

La estructura general persigue los siguientes propósitos:

- Centralizar la visualización y gestión de información geoespacial en una sola interfaz de usuario accesible desde el navegador.
- Separar la capa de presentación, la lógica de negocio y la persistencia espacial para mejorar la mantenibilidad.
- Permitir la carga y transformación de datos georreferenciados en múltiples formatos de uso habitual en proyectos SIG.
- Incorporar mecanismos de control de acceso y de revisión de contenidos antes de su publicación.
- Registrar evidencias de inserciones, modificaciones y eliminaciones mediante un esquema formal de auditoría.
- Ofrecer una base tecnológica suficientemente general como para ser reutilizada en otros estudios de aguas subterráneas, recursos hídricos o variables ambientales continuas.

1.3. Visión general por capas

Desde una perspectiva estructural, la aplicación puede describirse mediante cinco capas principales:

1. **Capa de presentación.** Corresponde a la interfaz visible por el usuario dentro del navegador. Esta capa está compuesta por una plantilla

HTML principal, estilos CSS, componentes modales, paneles flotantes, leyendas, formularios dinámicos y scripts JavaScript que controlan la interacción del mapa.

2. **Capa de interacción cartográfica.** Está formada por la biblioteca Leaflet y por los complementos usados para la visualización de puntos, superficies, capas raster y heatmaps. Esta capa resuelve la representación espacial en el cliente web.
3. **Capa de aplicación.** Implementada en Django, contiene la lógica de autenticación, permisos, carga de datos, transformación de coordenadas, publicación de capas, administración de usuarios y servicios de descarga.
4. **Capa de persistencia.** Corresponde a PostgreSQL con extensión PostGIS, donde se almacenan los puntos de muestreo, las capas vectoriales, las preferencias de usuario, la información de solicitudes, la auditoría y la referencia a archivos raster.
5. **Capa de procesamiento geoespacial auxiliar.** Incluye herramientas externas como `ogr2ogr`, `gdalinfo`, `gdalwarp`, `gdaldem` y `gdal_translate`, utilizadas para importar, reproyectar y preparar datos espaciales en formatos vectoriales y raster.

Estas capas no funcionan como componentes aislados, sino como partes coordinadas de un flujo de trabajo donde la información se captura, se valida, se transforma, se persiste y finalmente se representa de manera visual en el mapa.

1.4. Estructura física del proyecto

La organización del proyecto dentro del sistema de archivos refleja el patrón usual de una aplicación Django, pero adaptado a necesidades geoespaciales y a un proceso de documentación académica. A nivel de carpetas principales, la estructura observable es la siguiente:

- `tesis/`: contiene la configuración general del proyecto Django.
- `mapas/`: concentra la aplicación principal, incluyendo modelos, vistas, rutas, formularios, roles, migraciones, archivos estáticos y plantillas.
- `mapas/templates/mapas/`: contiene la interfaz principal del visor y las plantillas asociadas.

- `mapas/static/`: almacena recursos estáticos propios de la aplicación, tales como imágenes, logos y algunos raster de referencia.
- `staticfiles/`: corresponde al destino de archivos estáticos recolectados para despliegue.
- `sql/`: reúne scripts SQL auxiliares, especialmente aquellos necesarios para tablas no gestionadas por migraciones completas de Django.
- `libro/`: almacena archivos L^AT_EX y materiales vinculados al documento de tesis.
- `data/`: puede utilizarse como área de apoyo para datos crudos o archivos intermedios del proyecto.

Este orden no es meramente organizativo; también expresa una separación de responsabilidades entre configuración, código de aplicación, recursos estáticos, persistencia auxiliar y documentación.

1.5. Núcleo tecnológico de la plataforma

1.5.1. Framework de servidor

El backend está implementado con Django 5.1.6, que cumple el rol de framework principal para el manejo de rutas, vistas, validación, autenticación, renderizado de plantillas y acceso a la base de datos. La elección de Django responde a varias razones: estabilidad, madurez del ecosistema, facilidad para estructurar aplicaciones modulares y disponibilidad de extensiones geoespaciales a través de GeoDjango.

1.5.2. Capacidades geoespaciales

La aplicación utiliza `django.contrib.gis`, es decir, GeoDjango, para incorporar campos geométricos, serialización de geometrías y compatibilidad con PostGIS. Esto permite definir modelos con tipos espaciales como `PointField` y `MultiPolygonField`, y operar sobre ellos desde el backend sin abandonar el marco general de Django.

1.5.3. Base de datos espacial

La base de datos utilizada es PostgreSQL con extensión PostGIS. Esta combinación permite almacenar geometrías, ejecutar transformaciones de

coordenadas, calcular extensiones espaciales y persistir información alfanumérica y geográfica en un mismo repositorio. En la implementación actual, PostGIS no solo cumple la función de almacenamiento, sino también la de motor de transformación espacial para algunos procesos críticos, como la conversión de coordenadas de puntos importados desde CSV a EPSG:4326.

1.5.4. Biblioteca cartográfica en el cliente

En el frontend se utiliza **Leaflet**, una biblioteca JavaScript liviana y ampliamente difundida para mapas web. Sobre Leaflet se apoyan varios complementos y extensiones, entre ellos:

- `leaflet.heat`, para representar mapas de calor.
- Mecanismos de `imageOverlay` o equivalentes para capas raster derivadas.
- Integración con bibliotecas de lectura de raster en cliente, como `georaster` y `georaster-layer-for-leaflet`, según el modo de despliegue configurado.

1.5.5. Herramientas auxiliares de interfaz

La interfaz combina **Bootstrap** para la construcción de modales y formularios, y **SweetAlert2** para confirmaciones, advertencias, ingreso de comentarios, notificaciones de errores y mensajes de validación. Esto permite unificar la experiencia de usuario en operaciones sensibles como publicar capas, renombrar grupos, confirmar eliminaciones o cargar datos con advertencias.

1.5.6. Dependencias de despliegue

El proyecto ya contempla una base mínima para despliegue mediante:

- `gunicorn`, como servidor WSGI.
- `whitenoise`, para distribución de archivos estáticos en producción.
- Un `Dockerfile` orientado a contenedores, que instala dependencias del sistema como GDAL, GEOS, PROJ y bibliotecas de PostgreSQL.

Esto es particularmente importante en una plataforma SIG, donde no basta con desplegar solo paquetes Python, sino que también deben resolverse correctamente dependencias nativas de procesamiento geoespacial.

1.6. Configuración central del proyecto

La carpeta `tesis/` contiene la configuración general del proyecto Django, destacándose dos archivos principales.

1.6.1. `settings.py`

El archivo de configuración define los aspectos transversales de la aplicación:

- aplicaciones instaladas;
- middleware de seguridad, sesión, autenticación y CSRF;
- conexión a base de datos PostGIS;
- configuración de archivos estáticos y multimedia;
- soporte condicional para `WhiteNoise`;
- carga de variables de entorno para despliegue;
- configuración local de rutas SIG en Windows para `GDAL` y `PROJ`.

Un aspecto relevante es que la configuración no está totalmente fija a un entorno de desarrollo, sino que admite parametrización por variables de entorno, lo cual constituye una base necesaria para futuros despliegues en servidores o servicios cloud.

1.6.2. `urls.py`

El archivo principal de rutas delega la mayor parte de la funcionalidad a la aplicación `mapas`. Esta decisión implica que el dominio funcional del proyecto está claramente concentrado en una sola app principal, lo cual simplifica el mantenimiento mientras el sistema siga respondiendo a un ámbito problemático homogéneo.

1.7. Aplicación principal mapas

La aplicación `mapas` es el núcleo operativo del sistema. Dentro de ella se implementan los modelos de datos, las vistas, las rutas, los formularios, la lógica de permisos y la interfaz principal del visor.

1.7.1. Rutas funcionales

El archivo `mapas/urls.py` agrupa los servicios que componen la aplicación. Desde el punto de vista funcional, estas rutas pueden clasificarse en las siguientes categorías:

1. **Acceso y sesión:** inicio de sesión, cierre de sesión, registro y recuperación de contraseña.
2. **Visor principal:** carga del mapa de muestreo.
3. **Puntos de muestreo:** guardado manual, edición, eliminación, carga masiva por CSV y gestión de grupos de puntos.
4. **Capas vectoriales:** carga de GeoJSON, carga de Shapefile, eliminación y publicación.
5. **Capas raster:** previsualización de TIFF, carga de TIFF y eliminación de raster.
6. **Solicitudes de publicación:** envío, listado, aprobación y rechazo.
7. **Administración:** listado de usuarios, promoción y revocación de privilegios.
8. **Descargas:** exportación de puntos, capas y raster.

Esta segmentación muestra que la aplicación no se reduce a una sola página con funciones accesorias, sino que expone un conjunto relativamente amplio de servicios HTTP coherentes con el dominio de un sistema de información geoespacial colaborativo.

1.8. Modelado de datos

El diseño de datos del sistema se apoya en varios modelos principales, cada uno asociado a un tipo de entidad funcional.

1.8.1. Modelo Muestreo

El modelo **Muestreo** representa un punto de muestreo de calidad de agua. Este es el modelo más extenso de la aplicación, pues almacena tanto la ubicación espacial del punto como sus atributos físico-químicos y microbiológicos. Entre sus características se destacan:

- identificación del punto mediante códigos de pozo y otros campos alfanuméricos;
- coordenadas almacenadas como geometría puntual y también en columnas numéricas;
- fecha de toma;
- variables como nitratos, nitritos, alcalinidad, magnesio, conductividad, pH, coliformes fecales y otros parámetros adicionales;
- asociación a un usuario propietario;
- nombre de grupo;
- estado activo;
- visibilidad pública o privada;
- metadatos de lote de carga, archivo origen y SRID de origen.

La presencia de un número elevado de campos responde a la necesidad de manejar conjuntos de datos históricos heterogéneos, donde distintos muestreos pueden registrar distintas combinaciones de variables. El modelo fue adaptado para preservar esta riqueza de atributos y permitir tanto el análisis puntual como la carga progresiva de nuevas campañas.

Debe señalarse además que este modelo está marcado como `managed = False`, lo que implica que su tabla principal existe previamente en la base de datos y no es administrada completamente por migraciones automáticas de Django. Esta decisión suele adoptarse cuando la estructura de datos ya existía o cuando se busca compatibilidad con esquemas heredados.

1.8.2. Modelo Capa

El modelo **Capa** representa una capa vectorial superficial almacenada en la tabla `patino`. Se utiliza principalmente para polígonos o multipolígonos, como el límite del acuífero u otras coberturas espaciales cargadas por usuarios. Sus componentes principales son:

- geometría multipoligonal en `EPSG:4326`;
- nombre y descripción;
- usuario propietario, cuando la capa no es pública;

- estado funcional de la capa, tal como privada, pendiente, pública o rechazada.

Al igual que **Muestreo**, este modelo también trabaja sobre una tabla no gestionada enteramente por Django.

1.8.3. Modelo PreferenciasMapa

Este modelo almacena una preferencia espacial simple: el centro del mapa asociado a un usuario. Su función es mejorar la experiencia de uso, recordando el último centro relevante o preferido para el usuario autenticado.

1.8.4. Modelo CapaRaster

El modelo **CapaRaster** registra capas raster cargadas por usuarios, especialmente aquellas provenientes de archivos TIFF o GeoTIFF. Almacena:

- nombre del raster;
- usuario propietario;
- indicador de publicidad;
- modo de despliegue;
- ruta al archivo original;
- ruta al archivo reproyectado a EPSG:4326;
- ruta al archivo PNG derivado;
- límites geográficos;
- metadatos del raster.

Esto evidencia que el sistema no trata el raster como una simple imagen, sino como un recurso geoespacial completo que puede ser mostrado en distintos modos de visualización.

1.8.5. Modelo SolicitudPublicacion

La publicación de capas y grupos de puntos se gestiona mediante solicitudes formales. Este modelo registra:

- solicitante;
- tipo de solicitud;
- referencia a la capa o al grupo de puntos;
- estado de la solicitud;
- comentario de revisión;
- usuario revisor;
- fechas de creación, revisión y actualización.

Su existencia introduce un flujo de moderación explícito, evitando que cualquier usuario convierta unilateralmente sus recursos en contenido público sin intervención administrativa.

1.8.6. Modelo AuditoriaEvento

La auditoría centralizada se implementa mediante el modelo **AuditoriaEvento**. Esta entidad registra evidencia estructurada de inserciones, actualizaciones y eliminaciones. Sus campos permiten conocer:

- qué usuario ejecutó la acción;
- qué entidad fue afectada;
- cuál fue el identificador del registro afectado;
- cuál era el estado anterior y posterior;
- cuál fue la ruta y el método HTTP;
- desde qué dirección IP se realizó la acción;
- en qué momento se produjo el evento.

La incorporación de esta tabla transforma la aplicación en un sistema con trazabilidad operativa, una propiedad especialmente relevante cuando existen cargas masivas, moderación de contenidos y múltiples usuarios interactuando sobre el mismo entorno espacial.

1.9. Mezcla de tablas gestionadas y no gestionadas

Una característica importante de esta aplicación es que combina modelos totalmente gestionados por migraciones de Django con modelos asociados a tablas preexistentes o parcialmente heredadas. Esto se traduce en dos estrategias de administración de esquema:

- **tablas gestionadas por Django**, por ejemplo `PreferenciasMapa`, `CapaRaster`, `SolicitudPublicacion` y `AuditoriaEvento`;
- **tablas no gestionadas completamente por Django**, como `muestreo` y `patino`, donde la estructura principal proviene de un esquema ya existente y ciertas ampliaciones deben acompañarse de scripts SQL específicos.

Desde el punto de vista metodológico, esta situación ilustra un escenario frecuente en sistemas institucionales: la necesidad de integrar aplicaciones nuevas con bases de datos heredadas, sin imponer una reconstrucción completa del modelo de datos original.

1.10. Lógica de roles y permisos

La plataforma define distintos niveles de uso, asociados a capacidades diferenciadas.

1.10.1. Invitado

El usuario no autenticado puede acceder al visor, visualizar capas públicas y grupos de puntos públicos, descargar contenidos públicos y utilizar funcionalidades de consulta y filtrado. No puede modificar datos, cargar información ni administrar contenidos.

1.10.2. Usuario autenticado

El usuario autenticado puede:

- cargar puntos manualmente o por CSV;
- editar y eliminar sus propios puntos;
- crear y administrar grupos de puntos propios;

- cargar capas vectoriales y raster;
- solicitar la publicación de sus recursos;
- descargar sus propios recursos además de los públicos.

1.10.3. Administrador de mapas

El administrador de mapas está vinculado al grupo `map_admin` y, en ciertos flujos, a atributos administrativos de Django. Este rol puede:

- listar usuarios;
- otorgar o revocar privilegios de administración;
- visualizar todas las capas;
- resolver solicitudes de publicación;
- consultar datos ampliados de auditoría o procedencia.

1.10.4. Superusuario

El superusuario conserva el nivel más alto de privilegios dentro del ecosistema Django. En la lógica del proyecto, también es reconocido como administrador de mapas.

1.11. Interfaz principal del visor

La interfaz del sistema está concentrada principalmente en la plantilla `mapa_muestreo.html`. Este archivo actúa como una interfaz de tipo *single-page application* liviana, donde una gran parte de las interacciones del usuario se resuelven sin abandonar la página principal.

1.11.1. Elementos estructurales de la interfaz

Los elementos principales de la interfaz incluyen:

- cabecera superior con identidad visual, control de modo noche e inicio o cierre de sesión;
- mapa principal Leaflet;

- panel lateral de capas;
- menú flotante de acciones;
- múltiples modales para carga, edición, previsualización, filtrado, autenticación, solicitudes y administración.

La concentración de estas funciones en una sola vista responde a una estrategia de experiencia de usuario donde la mayor parte de las acciones se realiza en continuidad visual sobre el mapa, minimizando transiciones innecesarias entre páginas.

1.11.2. Panel de capas

El panel lateral de capas fue diseñado para soportar una jerarquía funcional, no solo visual. En su configuración actual, organiza la información en bloques tales como:

- grupos propios de puntos;
- capas propias vectoriales;
- capas propias raster;
- grupos de puntos públicos;
- capas públicas;
- capas base.

Adicionalmente, el panel soporta:

- colapsado y expansión de secciones;
- personalización de color de grupos y capas;
- reordenamiento visual de capas;
- activación y desactivación de visibilidad;
- sincronización entre estados lógicos y representación cartográfica.

1.11.3. Menú flotante

El menú flotante concentra operaciones orientadas a la gestión de datos y a la administración del visor. Entre sus opciones se incluyen la carga de puntos, la carga de capas, la carga de TIFF, la gestión de grupos, el filtrado, la consulta de descargas y funcionalidades administrativas. La decisión de agrupar estas acciones en un menú flotante permite ahorrar espacio en pantalla y mantener el mapa como componente visual central.

1.12. Gestión de puntos de muestreo

1.12.1. Carga manual de un punto

La carga manual permite crear un punto nuevo desde la interfaz. El usuario puede definir atributos descriptivos, variables físico-químicas y microbiológicas, y la ubicación espacial. La ubicación puede establecerse de dos maneras:

- seleccionando directamente sobre el mapa;
- ingresando coordenadas manualmente en formato geográfico.

Esta doble modalidad responde a diferentes escenarios operativos: usuarios que trabajan explorando sobre el mapa y usuarios que ya disponen de coordenadas precisas.

1.12.2. Edición de puntos

Los puntos propios pueden ser editados por su propietario. La edición incluye atributos descriptivos, grupo, fecha, parámetros de calidad y ubicación espacial. Cuando la ubicación cambia, la aplicación solicita confirmación explícita para evitar modificaciones accidentales.

1.12.3. Carga masiva por CSV

La carga masiva de puntos es uno de los componentes más relevantes de la aplicación. Este flujo contempla:

1. selección del archivo;
2. indicación del grupo de puntos;
3. selección del sistema de coordenadas de origen;

4. lectura y previsualización del contenido;
5. detección de filas vacías estructurales;
6. mapeo flexible de encabezados mediante alias normalizados;
7. validación preliminar de parámetros críticos para cálculo de WQI;
8. detección automática de posibles inconsistencias en el SRID;
9. conversión de coordenadas a EPSG:4326 para visualización;
10. inserción transaccional de los puntos.

La carga CSV no se limita a leer columnas fijas. El sistema admite variantes de encabezados y reconoce distintos nombres equivalentes para las mismas variables. Esta estrategia es esencial cuando se trabaja con series históricas provenientes de campañas, laboratorios o planillas de origen heterogéneo.

1.12.4. Conversión de coordenadas

Un aspecto crítico del flujo CSV es la interpretación de coordenadas. El usuario selecciona un SRID de origen, por ejemplo EPSG:32721 o EPSG:4326. A partir de esta definición, el sistema transforma los puntos a EPSG:4326 para garantizar que puedan representarse en el mapa web. Además, se incluye una verificación heurística del rango de coordenadas para detectar cuando el archivo parece haber sido clasificado con un SRID erróneo.

Este detalle es metodológicamente importante: la aplicación no solo almacena coordenadas, sino que también controla su consistencia espacial para evitar inserciones invisibles o mal geolocalizadas.

1.12.5. WQI y disponibilidad de parámetros

La plataforma contempla un cálculo de WQI condicionado por la disponibilidad de un conjunto mínimo de parámetros críticos. Cuando un punto no cumple con el umbral definido, la aplicación no bloquea necesariamente la inserción, pero emite advertencias para informar que ese registro no podrá ser utilizado inmediatamente para el cálculo del indicador.

Este comportamiento muestra una decisión de diseño orientada a la trazabilidad de datos incompletos: el sistema no rechaza automáticamente registros potencialmente valiosos, pero deja constancia de su limitación analítica.

1.13. Gestión de grupos de puntos

Los grupos de puntos constituyen una abstracción funcional que permite organizar muestreos pertenecientes a una misma campaña, serie temporal, origen institucional o unidad temática. La aplicación permite:

- asignar un grupo al insertar puntos;
- renombrar grupos propios;
- cambiar su visibilidad;
- eliminar un grupo completo;
- personalizar su color en el mapa y en el panel lateral;
- solicitar su publicación al administrador.

En términos de arquitectura, esto significa que la plataforma no gestiona solo puntos individuales, sino colecciones semánticas de puntos, lo cual mejora la usabilidad y simplifica operaciones masivas.

1.14. Gestión de capas vectoriales

1.14.1. Carga de GeoJSON

La aplicación admite la carga de geometrías en formato GeoJSON. El backend valida la sintaxis del archivo, reconoce si se trata de una **FeatureCollection**, una **Feature** o una geometría suelta, y transforma polígonos simples en multipolígonos cuando corresponde. Esto permite una incorporación flexible de capas poligonales sin requerir una preparación excesiva del archivo de origen.

1.14.2. Carga de Shapefile

La carga de Shapefile se implementa mediante un archivo ZIP que debe contener al menos los componentes **.shp**, **.shx**, **.dbf** y **.prj**. Una vez recibido, el sistema:

1. descomprime el archivo;
2. verifica que la estructura esté completa;
3. ejecuta **ogr2ogr** para importar a PostgreSQL;

4. reproyecta a EPSG:4326;
5. asigna el usuario propietario a las geometrías recién insertadas;
6. registra la operación en auditoría.

Este flujo demuestra la capacidad del sistema para integrarse con formatos clásicos de SIG de escritorio y llevarlos a un entorno web colaborativo.

1.14.3. Publicación de capas vectoriales

Las capas vectoriales cargadas por usuarios pueden permanecer privadas o solicitarse para publicación. El administrador evalúa la solicitud y puede aprobarla o rechazarla, agregando un comentario. En caso de aprobación, la capa pasa al conjunto de recursos visibles para invitados y usuarios generales.

1.15. Gestión de capas raster

1.15.1. Previsualización de TIFF

Antes de almacenar un raster, la aplicación permite analizar un archivo TIFF o GeoTIFF y extraer metadatos como tamaño, sistema de coordenadas, cantidad de bandas, tipo de dato, valor NoData, resolución y límites espaciales. Esta etapa evita cargas ciegas y mejora la comprensión del recurso antes de su incorporación al visor.

1.15.2. Normalización y despliegue

Una vez aceptado, el raster puede ser:

- reproyectado a EPSG:4326;
- coloreado mediante una rampa definida por archivo de relieve;
- convertido a PNG para una visualización rápida;
- mantenido como GeoTIFF procesado para usos técnicos posteriores.

El almacenamiento tanto del archivo original como de sus derivados constituye una estrategia de preservación del dato fuente y de adaptación para el consumo web.

1.16. Solicitudes de publicación y moderación

El sistema incorpora una capa de gobernanza de contenidos mediante el modelo de solicitudes de publicación. Esta funcionalidad es especialmente relevante porque introduce una distinción entre *cargar* y *publicar*. Un usuario puede generar y administrar sus datos, pero la exposición pública de los mismos requiere un paso adicional de revisión.

El administrador dispone de un modal para:

- listar solicitudes pendientes;
- revisar historial de solicitudes resueltas;
- filtrar por tipo y estado;
- aprobar o rechazar;
- registrar un comentario de decisión.

Esta estructura es equivalente, a pequeña escala, a un flujo editorial o de moderación de contenidos, lo que aporta control de calidad y claridad institucional sobre qué información pasa a ser pública.

1.17. Mecanismos de descarga

La aplicación permite descargar diferentes recursos en formatos adecuados a su naturaleza:

- puntos en CSV o GeoJSON;
- grupos de puntos en CSV o GeoJSON;
- capas vectoriales en GeoJSON;
- raster en GeoTIFF o PNG.

La lógica de permisos distingue recursos propios, públicos y no accesibles. De este modo, la plataforma no solo funciona como visor y editor, sino también como repositorio de intercambio de datos geoespaciales.

1.18. Persistencia y transformación espacial

Una de las fortalezas centrales de la arquitectura es que la persistencia no se limita al almacenamiento bruto de archivos o filas, sino que incorpora transformaciones y validaciones espaciales. Entre las operaciones más relevantes se encuentran:

- transformación de coordenadas de origen a EPSG:4326;
- serialización GeoJSON para consumo del frontend;
- reproyección de raster;
- incorporación de geometrías desde OGR a PostGIS;
- cálculo de extensiones y metadatos raster.

Esto ubica a la base de datos y a las utilidades GDAL/OGR como componentes activos del comportamiento del sistema, y no como meros repositorios pasivos.

1.19. Auditoría y trazabilidad

La incorporación del modelo `AuditoriaEvento` permite documentar operaciones de inserción, actualización y eliminación sobre entidades clave de la aplicación. Este mecanismo registra:

- actor de la acción;
- entidad afectada;
- identificador del registro;
- estado previo;
- estado posterior;
- ruta HTTP;
- método HTTP;
- dirección IP de origen;
- metadatos de contexto, como origen de carga o motivo de la acción.

La auditoría implementada corresponde a una trazabilidad de nivel aplicación. Esto significa que registra cambios efectuados a través del sistema web. Si en el futuro se desea auditar también modificaciones directas realizadas desde herramientas externas sobre la base de datos, sería necesario complementar esta capa con disparadores (*triggers*) SQL.

1.20. Seguridad funcional

La seguridad de la plataforma se apoya en varios mecanismos complementarios:

- autenticación de usuarios mediante Django;
- protección CSRF en operaciones sensibles;
- restricción de operaciones según propietario del recurso;
- verificación de roles administrativos;
- separación entre recursos privados y públicos;
- moderación previa de contenidos publicables.

Si bien se trata de una aplicación académica en desarrollo, la presencia de estas medidas muestra una preocupación concreta por el control de integridad y la gestión responsable de la información espacial.

1.21. Desempeño y escalabilidad

La arquitectura actual fue diseñada para un escenario académico y de desarrollo progresivo. Aun así, ya incorpora elementos que favorecen la escalabilidad:

- almacenamiento estructurado en PostGIS;
- separación entre recursos vectoriales, raster y puntos;
- posibilidad de parametrizar el entorno de despliegue;
- soporte inicial para contenerización con Docker;
- uso de formatos abiertos y ampliamente interoperables.

No obstante, también existen desafíos técnicos asociados al crecimiento futuro, entre ellos:

- el tamaño y complejidad creciente de la plantilla principal del mapa;
- la concentración de lógica en una sola vista de gran extensión;
- la necesidad futura de separar más claramente componentes JavaScript, CSS y servicios API;
- la conveniencia de fortalecer pruebas automáticas de extremo a extremo.

Estas observaciones no invalidan la arquitectura, pero sí indican una posible hoja de ruta para refactorizaciones futuras.

1.22. Aporte metodológico de la estructura propuesta

Desde el punto de vista metodológico, la estructura de la aplicación presenta varias contribuciones relevantes:

- demuestra la viabilidad de construir un visor Web-GIS temático mediante herramientas abiertas;
- integra datos de muestreo con geometrías vectoriales y superficies raster en un mismo entorno operativo;
- incorpora mecanismos de colaboración y gobernanza de contenidos;
- documenta y registra la trazabilidad de las operaciones;
- permite adaptar el sistema a otros dominios territoriales o ambientales.

En consecuencia, la aplicación no debe interpretarse solo como un producto de software, sino como una propuesta de estructura funcional para la gestión geoespacial de datos ambientales en contextos académicos e institucionales.

1.23. Sugerencias de figuras a incorporar

Para enriquecer esta sección del documento, se recomienda incorporar posteriormente figuras explicativas. Algunas opciones particularmente útiles son las siguientes:

- diagrama general de arquitectura por capas;
- esquema entidad–relación simplificado de los modelos principales;
- captura del visor principal con identificación de sus componentes;
- flujo de carga masiva de puntos por CSV;
- flujo de revisión y publicación de capas o grupos;
- esquema del procesamiento de un GeoTIFF desde archivo original hasta despliegue web;
- diagrama del circuito de auditoría.

1.24. Conclusión de la sección

La estructura de la aplicación web desarrollada para el acuífero Patiño responde a una combinación de necesidades de visualización, gestión de datos, interoperabilidad geoespacial, colaboración multiusuario y trazabilidad. El sistema articula de manera coherente una base de datos espacial, un framework web robusto, bibliotecas de cartografía en cliente y herramientas auxiliares de procesamiento raster y vectorial.

Su diseño actual demuestra que es posible construir una plataforma especializada con herramientas abiertas, manteniendo capacidades avanzadas como importación de múltiples formatos, publicación moderada, personalización visual, exportación de datos y auditoría de acciones. Esta estructura no solo da soporte a los objetivos operativos del proyecto, sino que también constituye una base metodológica reutilizable para desarrollos similares orientados a la gestión de información geoespacial ambiental.